Week 6 (CS 418 @ UIC)

Week 6 Slide Deck

Visualizations

Jack Bandy 2026


Madison Street
and buses
behind you
Midway
Kimball
Washington/Wabash
Millennium Park
W
Washington/
Wabash

Demo Content Slide

Content for Week 6.

- Visualization refinement
- Interaction
- Time series
- Communicating uncertainty
- Sneak peek at modeling

Gapminder

embedded web page: https://www.gapminder.org/tools/#$chart-type=bubbles&url=v2

Time Series Visualization

Time Series Basics

A **univariate time series** is simply a sequence of observations of the **same variable** collected over time - not just a “bag of values”

Two goals of time series analysis:

- **Understand** or explain processes that produced the data
- **Forecast** future values of the series (i.e. prediction)

Time Series Examples

- Airline passengers
- Public transit ridership
- Hourly/Daily/Monthly temperatures
- Annual spending
- Others you can think of?

Why I love time series

Time series can contextualize (almost) any arbitrary datum

- e.g. “UIC had XXXX graduates last year”
- e.g. “Chicago’s budget grew by $XXmillion last year”
- e.g. “the mortgate interest rate is X.X%”
- e.g. “XX people have died from the flu this year”

Airline Passengers: Setup

```
1 import polars as pl
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4
5 airline = pl.read_csv("../../datasets/airline-passengers/airline-passengers.csv")
6 airline.sample(5, seed=42).sort("Month")
```

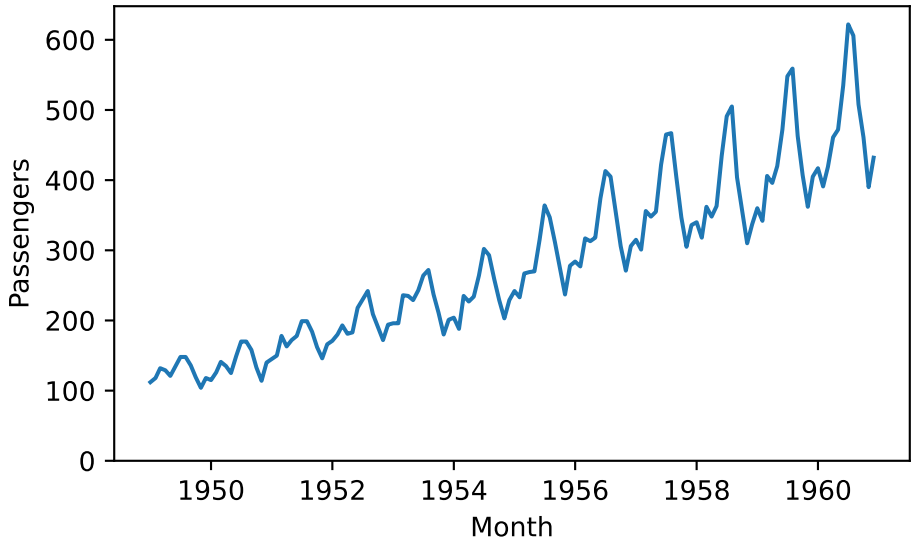
shape: (5, 2)

Month	Passengers
str	i64
"1952-09-01"	209
"1957-05-01"	355
"1958-07-01"	491
"1960-08-01"	606

"1960-12-01" 432

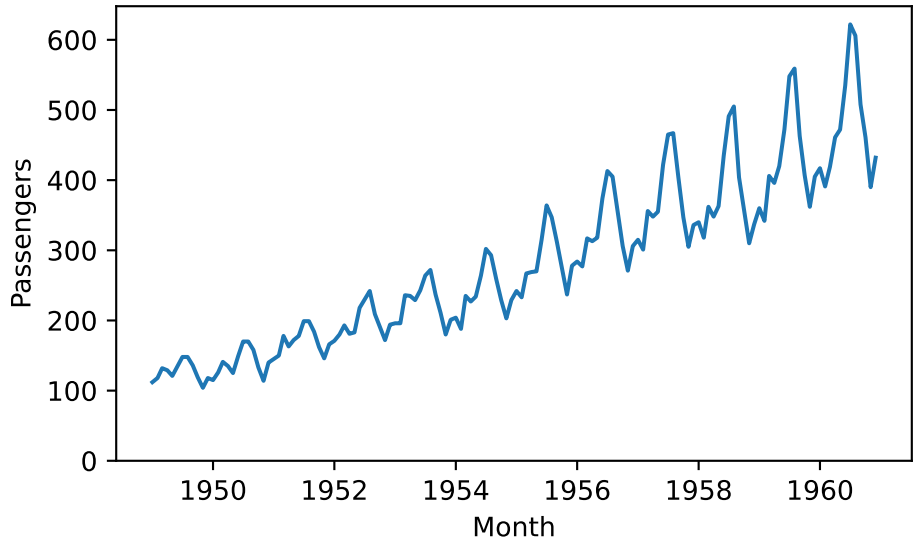
Airline Passengers Example

```
airline = airline.with_columns(pl.col("Month").str.to_date("%Y-%m-%d"))
ax = sns.lineplot(x=airline["Month"], y=airline["Passengers"])
ax.set_ylim(bottom=0)
plt.tight_layout()
```



Airline Passengers: What Do You Notice?

- How to describe this series?
- Overall trend(s)?
- Seasonal changes?
- Potential explanations?



Rolling Average

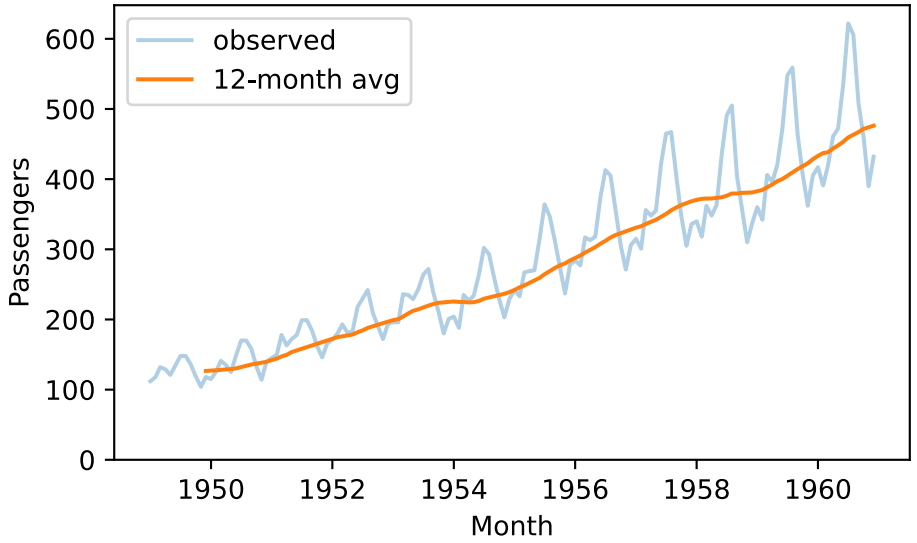
```
airline = airline.with_columns(
    pl.col("Passengers").rolling_mean(window_size=12).alias("trend")
)
airline.sample(5, seed=42).sort("Month")
```

shape: (5, 3)

Month	Passengers	trend
date	i64	f64
1952-09-01	209	190.083333
1957-05-01	355	342.083333
1958-07-01	491	376.333333
1960-08-01	606	463.333333
1960-12-01	432	476.166667

Rolling Average

```
1 ax = sns.lineplot(x=airline["Month"], y=airline["Passengers"], alpha=0.35, label="observed")
2 trend_not_null = airline.filter(pl.col("trend").is_not_null())
3 sns.lineplot(x=trend_not_null["Month"], y=trend_not_null["trend"], label="12-month avg", ax=ax)
4 ax.legend()
5 ax.set_ylim(bottom=0)
6 plt.tight_layout()
```



Four-part Decomposition

Time series can be decomposed into four parts:

- **Level** — Baseline average value
- **Trend** — Long-term increase or decrease
- **Seasonality** — Repeating periodic pattern(s)
- **Noise** — Random/unexplained variation

Additive vs. Multiplicative Seasonality

Additive

$$y(t) = T + S + N$$

Seasonal swings are **constant**: regardless of trend level, variation is a fixed amount.

→ *Use when amplitudes stay flat.*

Multiplicative

$$y(t) = T \times S \times N$$

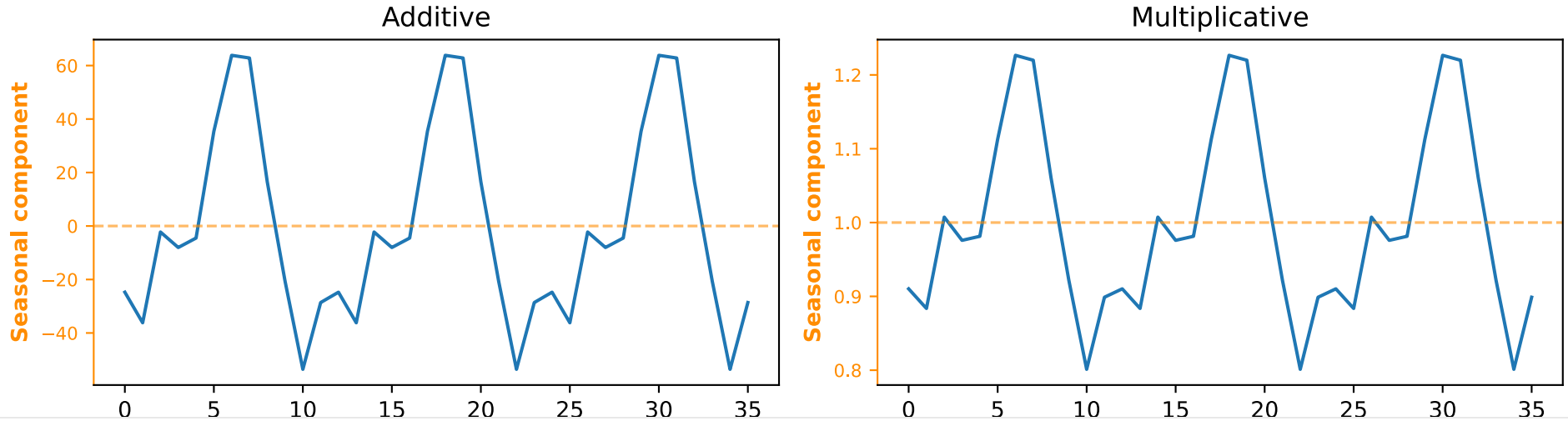
Seasonality **scales as the trendline continues** (bigger baseline means bigger swings).

→ *Use when amplitudes grow.*

(Basically, if peaks get taller as the series continues, use multiplicative.)

Additive vs. Multiplicative: Side by Side

```
from statsmodels.tsa.seasonal import seasonal_decompose
fig, axes = plt.subplots(1, 2, figsize=(10, 3.09))
for model, ax in zip(["additive", "multiplicative"], axes):
    r = seasonal_decompose(airline["Passengers"].to_numpy().astype(float), model=model, period=12)
    ax.plot(r.seasonal[:36])
    ax.set_title(f"{model.capitalize()}")
    ax.set_xlabel("Month (first 3 years)")
    ax.set_ylabel("Seasonal component", color="darkorange", fontweight="bold")
    ax.axhline(0 if model == "additive" else 1, color="darkorange", linewidth=1.2, linestyle="--", alpha=0.6)
    ax.tick_params(axis="y", colors="darkorange", labelsize=8)
    ax.spines["left"].set_color("darkorange")
plt.tight_layout()
```



Choosing the Right Seasonality Model

Use additive when seasonal swings are consistently the same size.

- *Example: a stadium closes outdoor seating during the winter, which reduces capacity by a fixed amount (e.g. 2,000 seats).*

Use multiplicative when seasonal swings scale with the trend.

- *Example: airline passengers — summer peaks are larger as the overall baseline gets higher (more people are flying).*

Remember to plot the raw data, check if the peaks get taller.

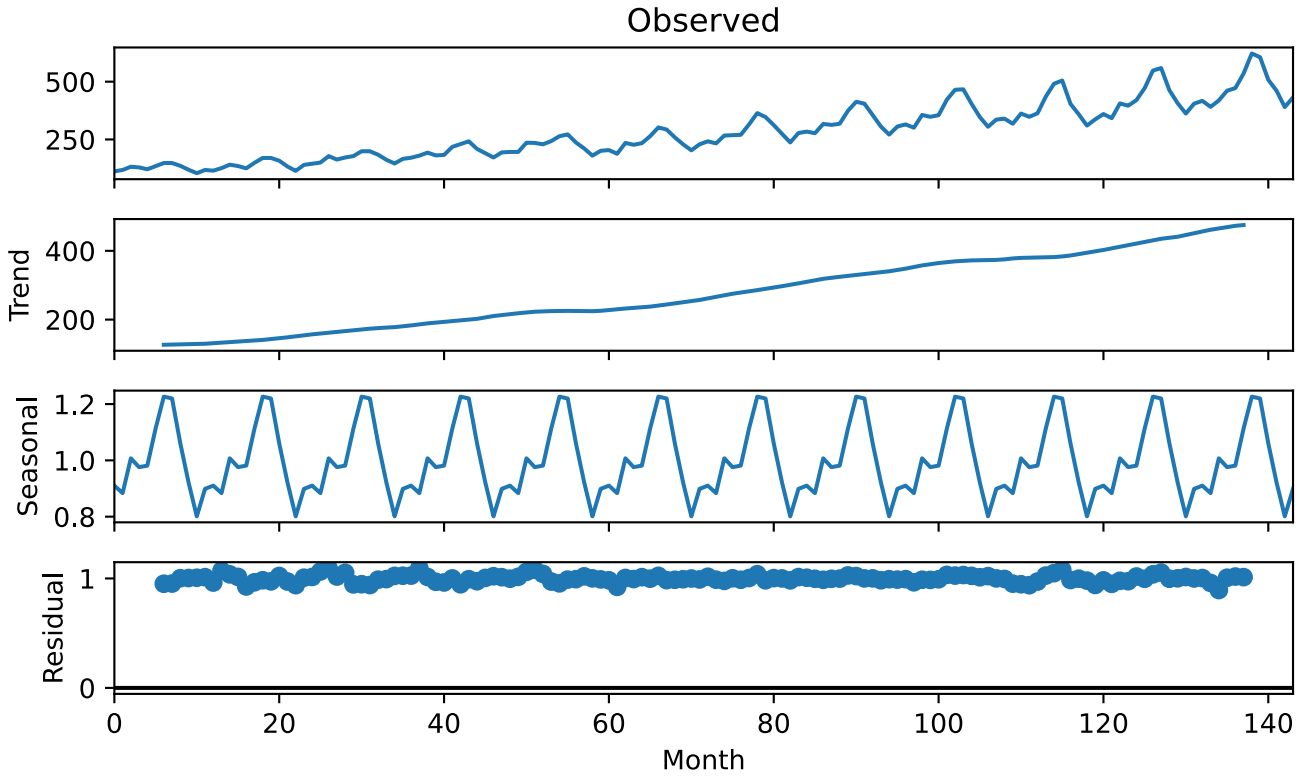
`seasonal_decompose`: Four Components

```
from statsmodels.tsa.seasonal import seasonal_decompose

values = airline["Passengers"].to_numpy().astype(float)
result = seasonal_decompose(values, model="multiplicative", period=12)
```

`seasonal_decompose`: Four Components

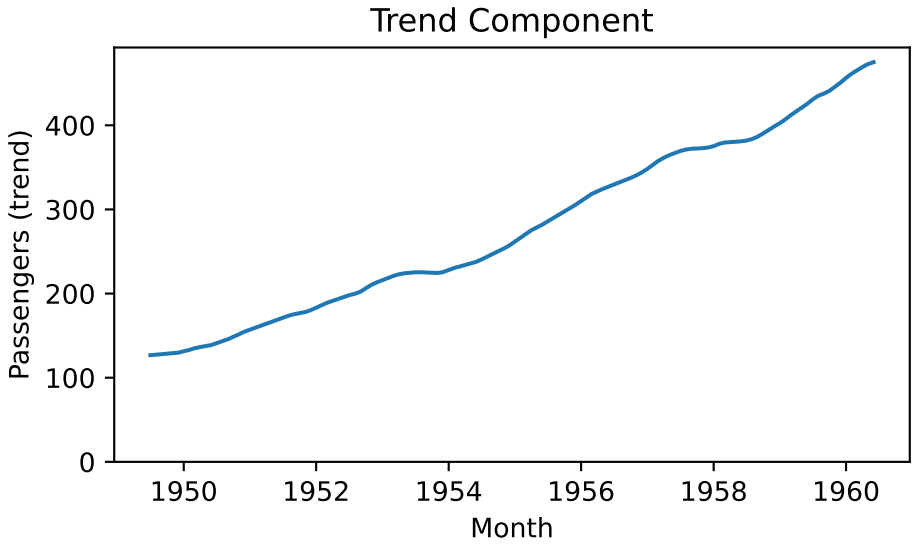
```
1 fig = result.plot()
2 fig.set_size_inches(7, 4.3)
3 fig.get_axes()[-1].set_xlabel("Month")
4 plt.tight_layout()
```



The Trend Component

```python
trend_df = pl.DataFrame({
    "Month": airline["Month"],
    "trend": pl.Series("trend", result.trend).fill_nan(None),
}).drop_nulls("trend")

ax = sns.lineplot(x=trend_df["Month"], y=trend_df["trend"])
ax.set_ylim(bottom=0)
ax.set_ylabel("Passengers (trend)")
ax.set_title("Trend Component")
plt.tight_layout()
```



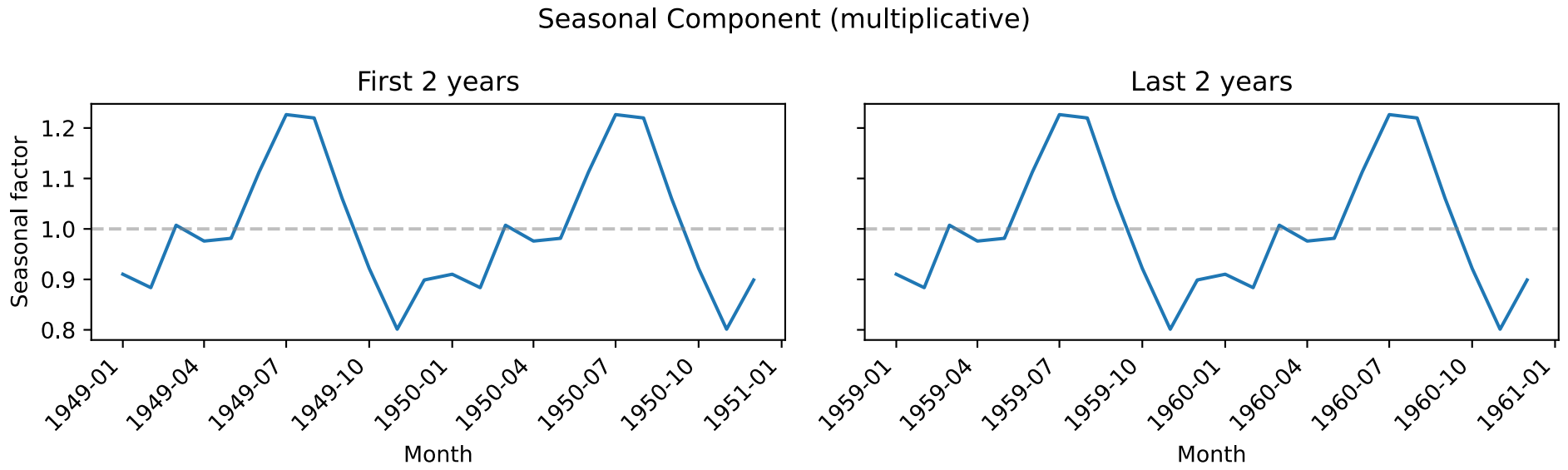
The Seasonal Component

```
seasonal_df = pl.DataFrame({
    "Month": airline["Month"],
    "seasonal": pl.Series("seasonal", result.seasonal),
})
first_two = seasonal_df.head(24)
last_two = seasonal_df.tail(24)
```

We'll check to see if the seasonal factor stays **constant** over time (with a multiplicative model)

The Seasonal Component: Stable Over Time?

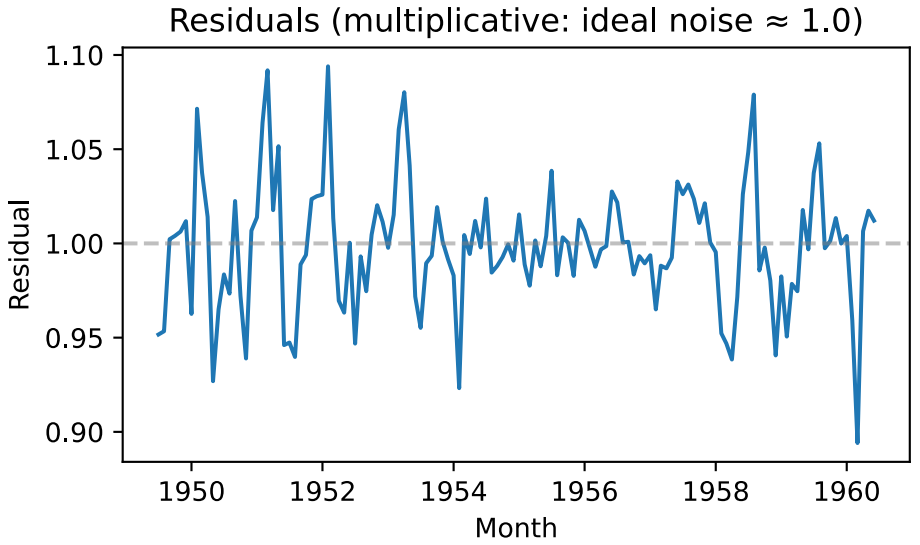
```
fig, axes = plt.subplots(1, 2, figsize=(10, 3.09), sharey=True)
for df, ax, label in zip([first_two, last_two], axes, ["First 2 years", "Last 2 years"]):
    sns.lineplot(x=df["Month"], y=df["seasonal"], ax=ax)
    ax.axhline(1.0, color="gray", linestyle="--", alpha=0.5)
    ax.set(xlabel="Month", title=label)
    plt.setp(ax.xaxis.get_majorticklabels(), rotation=45, ha="right")
axes[0].set_ylabel("Seasonal factor")
fig.suptitle("Seasonal Component (multiplicative)")
plt.tight_layout()
```



The Residuals

```
resid_df = pl.DataFrame({
    "Month": airline["Month"],
    "resid": pl.Series("resid", result.resid).fill_nan(None),
}).drop_nulls("resid")

ax = sns.lineplot(x=resid_df["Month"], y=resid_df["resid"])
ax.axhline(1.0, color="gray", linestyle="--", alpha=0.5)
ax.set_ylabel("Residual")
ax.set_title("Residuals (multiplicative: ideal noise ≈ 1.0)")
plt.tight_layout()
```



Stationarity

- Some forecasting models assume stationarity
- So you must know whether a series is stationary before modeling

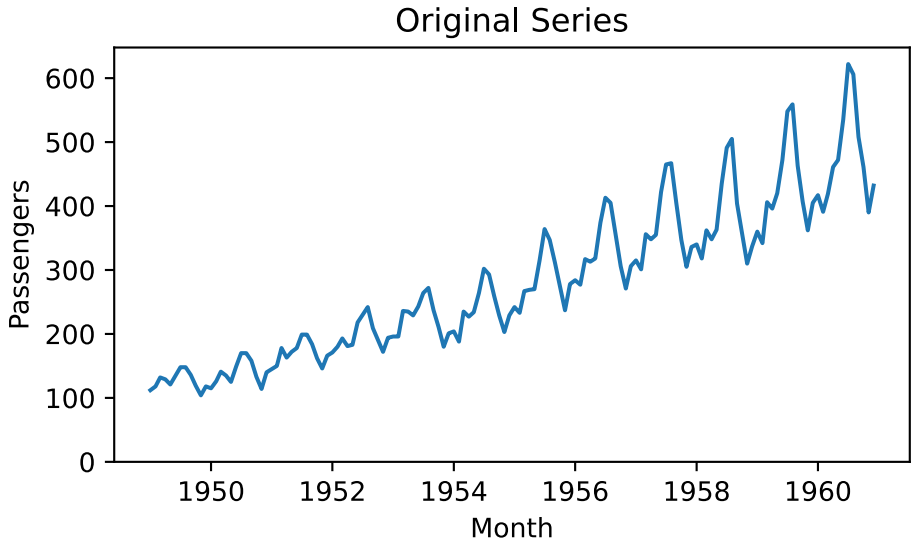
A stationary series is defined by:

- **Constant mean** — no long-term drift up or down
- **Constant variance** — spread stays the same over time
- **No trend or seasonality**

Stationary?

```
airline_diff = airline.with_columns(
    pl.col("Passengers").diff().alias("first_diff")
).drop_nulls()

ax = sns.lineplot(x=airline["Month"], y=airline["Passengers"])
ax.set_ylim(bottom=0)
ax.set_ylabel("Passengers")
ax.set_title("Original Series")
plt.tight_layout()
```



Making a Series Stationary: Differencing

Differencing removes trend by computing the difference between consecutive observations:

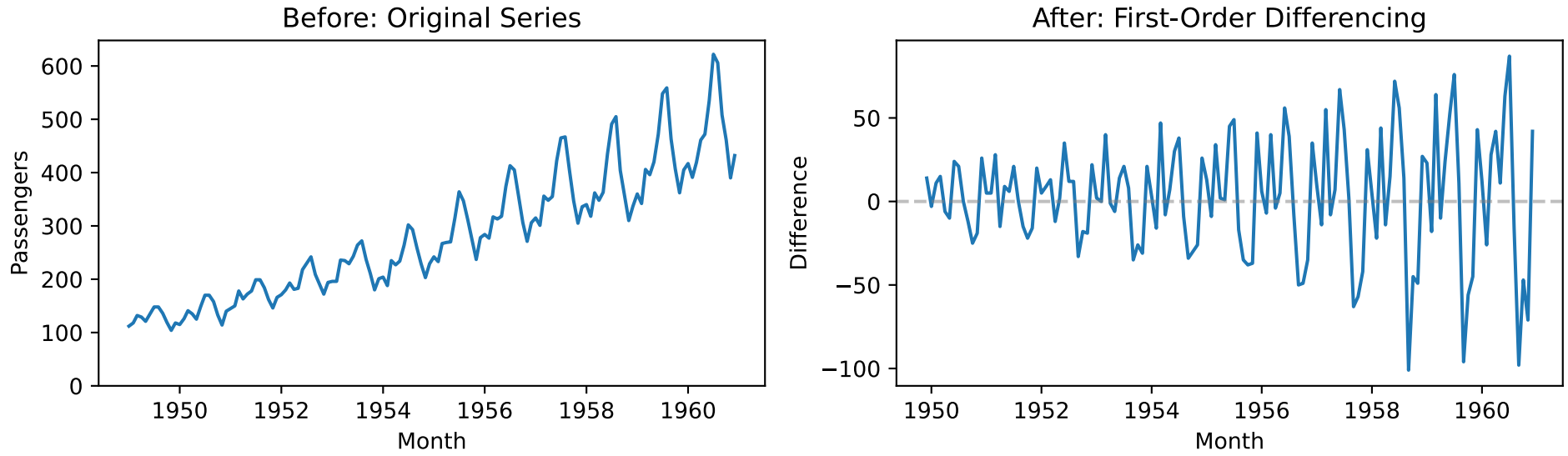
$$Y_t' = Y_t - Y_{t-1}$$

This is called **first-order differencing** (difference order = 1).

- (Recognize this concept?)
- Repeat the process for stronger trends
    - second-order differencing removes quadratic trends
- *Example: if passengers are 100 → 120 → 139, the first differences are 20, 19 (much flatter)*

Differencing: Before and After

```
fig, axes = plt.subplots(1, 2, figsize=(10, 3.09))
sns.lineplot(x=airline["Month"], y=airline["Passengers"], ax=axes[0])
axes[0].set_ylim(bottom=0); axes[0].set_title("Before: Original Series")
sns.lineplot(x=airline_diff["Month"], y=airline_diff["first_diff"], ax=axes[1])
axes[1].axhline(0, color="gray", linestyle="--", alpha=0.5)
axes[1].set_ylabel("Difference"); axes[1].set_title("After: First-Order Differencing")
plt.tight_layout()
```



Time Series Example: CTA Rides

Load UIC-Halsted Daily Ridership

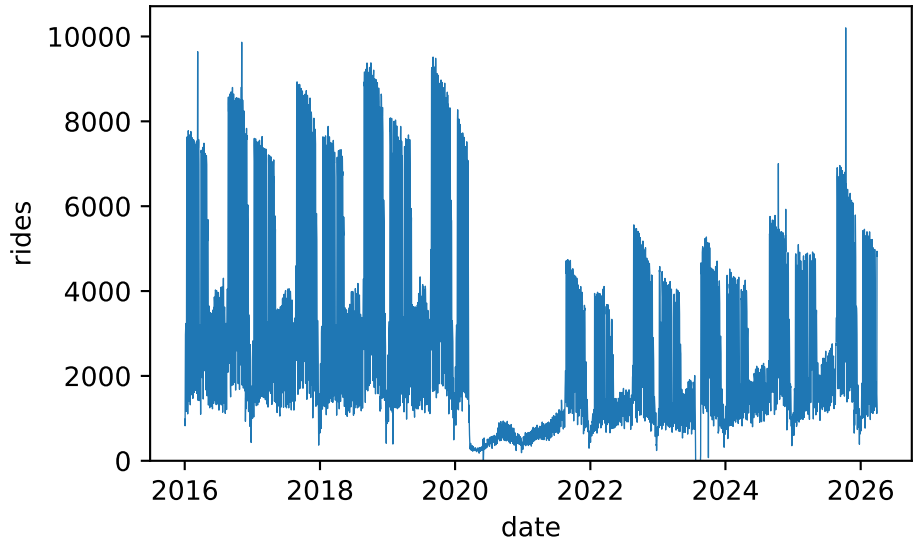
```
cta = pl.read_csv("../../datasets/cta-ridership/Station_Entries_-_Daily_Totals_20260527.csv")
uic = cta.filter(pl.col("stationname") == "UIC-Halsted")
uic.sample(5, seed=42)
```

shape: (5, 5)

station_id	stationname	date	daytype	rides
i64	str	str	str	str
40350	"UIC-Halsted"	"07/19/2021"	"W"	"1,068"
40350	"UIC-Halsted"	"01/19/2009"	"W"	"1,824"
40350	"UIC-Halsted"	"11/01/2025"	"A"	"1,811"
40350	"UIC-Halsted"	"09/12/2018"	"W"	"9,375"
40350	"UIC-Halsted"	"01/12/2021"	"W"	"566"

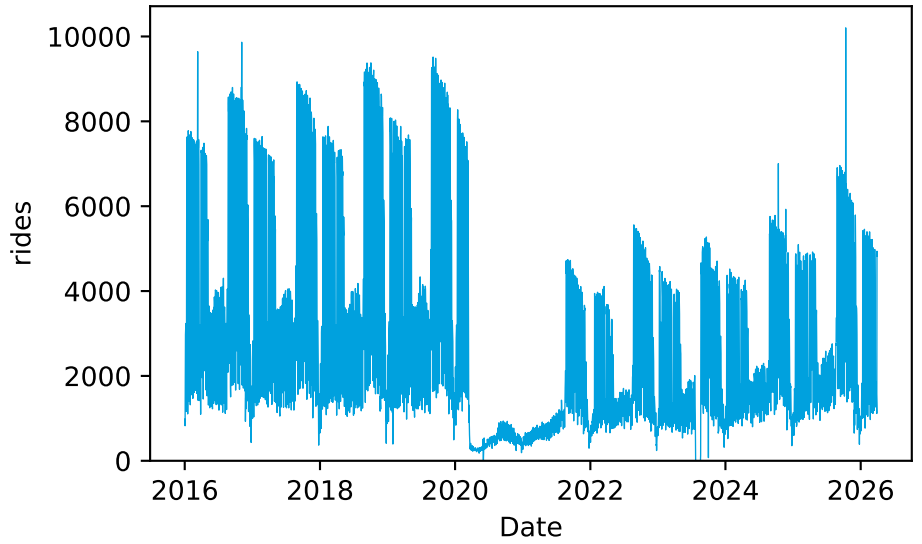
Load UIC-Halsted Daily Ridership

```python
uic = (
    cta.filter(pl.col("stationname") == "UIC-Halsted")
    .with_columns(
        pl.col("date").str.to_date("%m/%d/%Y"),
        pl.col("rides").str.replace_all(",", "").cast(pl.Int64),
    ).sort("date").filter(pl.col("date") >= pl.date(2016, 1, 1))
)
ax = sns.lineplot(x=uic["date"], y=uic["rides"], linewidth=0.5)
ax.set_ylim(bottom=0)
plt.tight_layout()
```



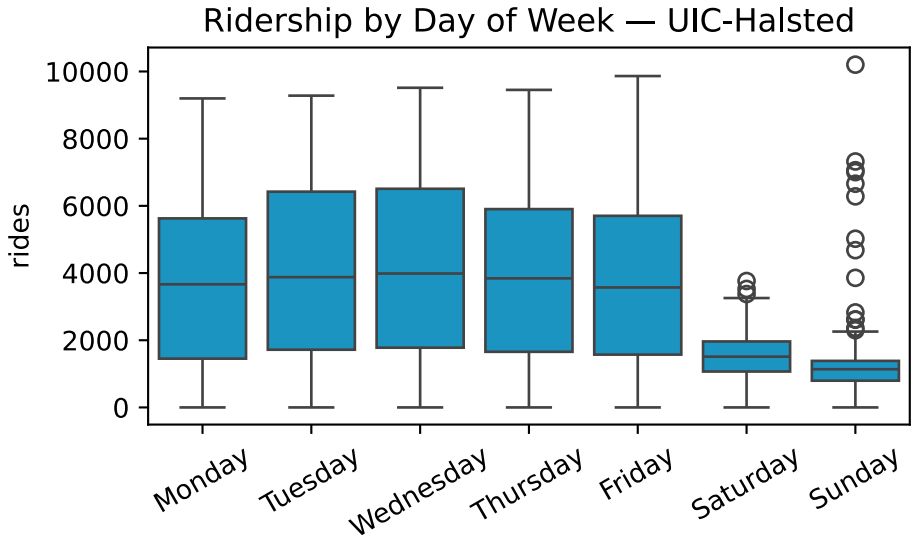
Just for Fun

```
CTA_BLUE = "#00A1DE"
ax = sns.lineplot(x=uic["date"], y=uic["rides"], linewidth=0.5, color=CTA_BLUE)
ax.set_xlabel("Date")
ax.set_ylim(bottom=0)
plt.tight_layout()
```



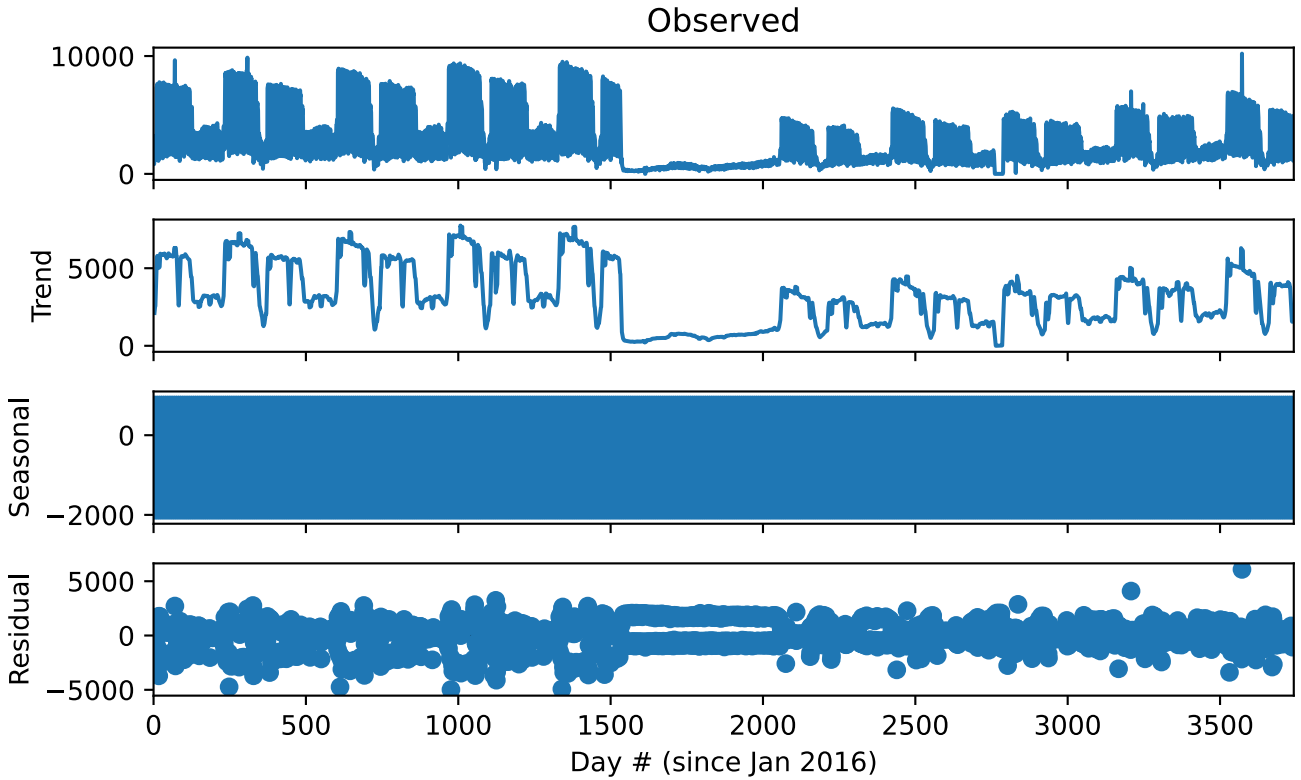
Weekday variation

```python
day_order = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
uic_dow = uic.with_columns(pl.col("date").dt.strftime("%A").alias("day_name"))
ax = sns.boxplot(x=uic_dow["day_name"], y=uic_dow["rides"], order=day_order, color=CTA_BLUE)
ax.set_xlabel("")
ax.tick_params(axis="x", rotation=30)
ax.set_title("Ridership by Day of Week — UIC-Halsted")
plt.tight_layout()
```



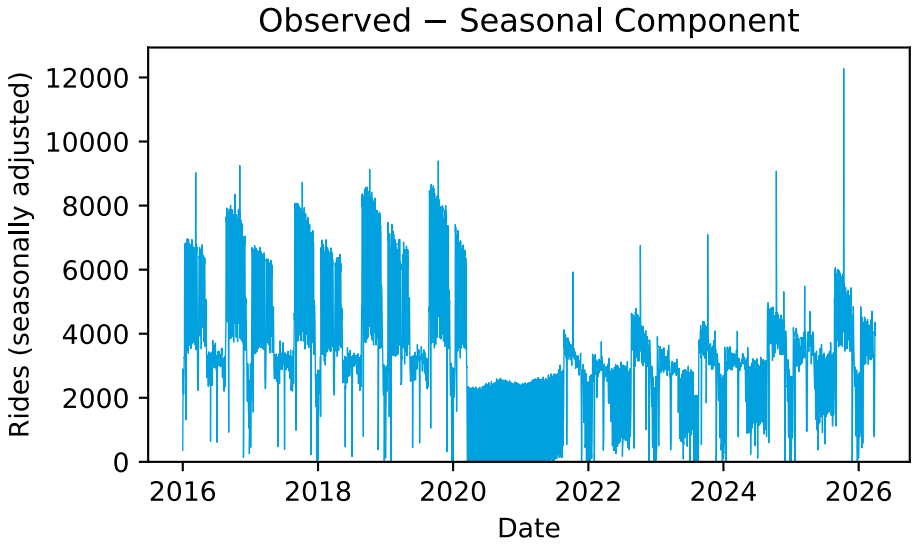
Decomposing with Period = 7

```
1 rides = uic["rides"].to_numpy().astype(float)
2 cta_result = seasonal_decompose(rides, model="additive", period=7)
3 fig = cta_result.plot()
4 fig.set_size_inches(7, 4.3)
5 fig.get_axes()[-1].set_xlabel("Day # (since Jan 2016)")
6 plt.tight_layout()
```



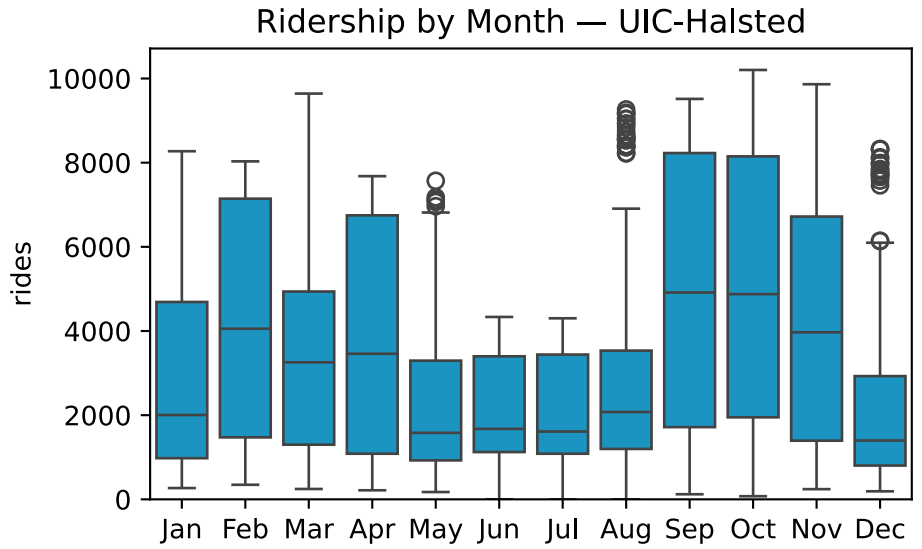
Period=7 Decomposition: What's Still There?

```python
deseasonalized = pl.DataFrame({
    "date": uic["date"],
    "rides": pl.Series("rides", cta_result.observed - cta_result.seasonal),
})
ax = sns.lineplot(x=deseasonalized["date"], y=deseasonalized["rides"], color=CTA_BLUE, linewidth=0.5)
ax.set_xlabel("Date")
ax.set_ylabel("Rides (seasonally adjusted)")
ax.set_title("Observed − Seasonal Component")
ax.set_ylim(bottom=0)
plt.tight_layout()
```



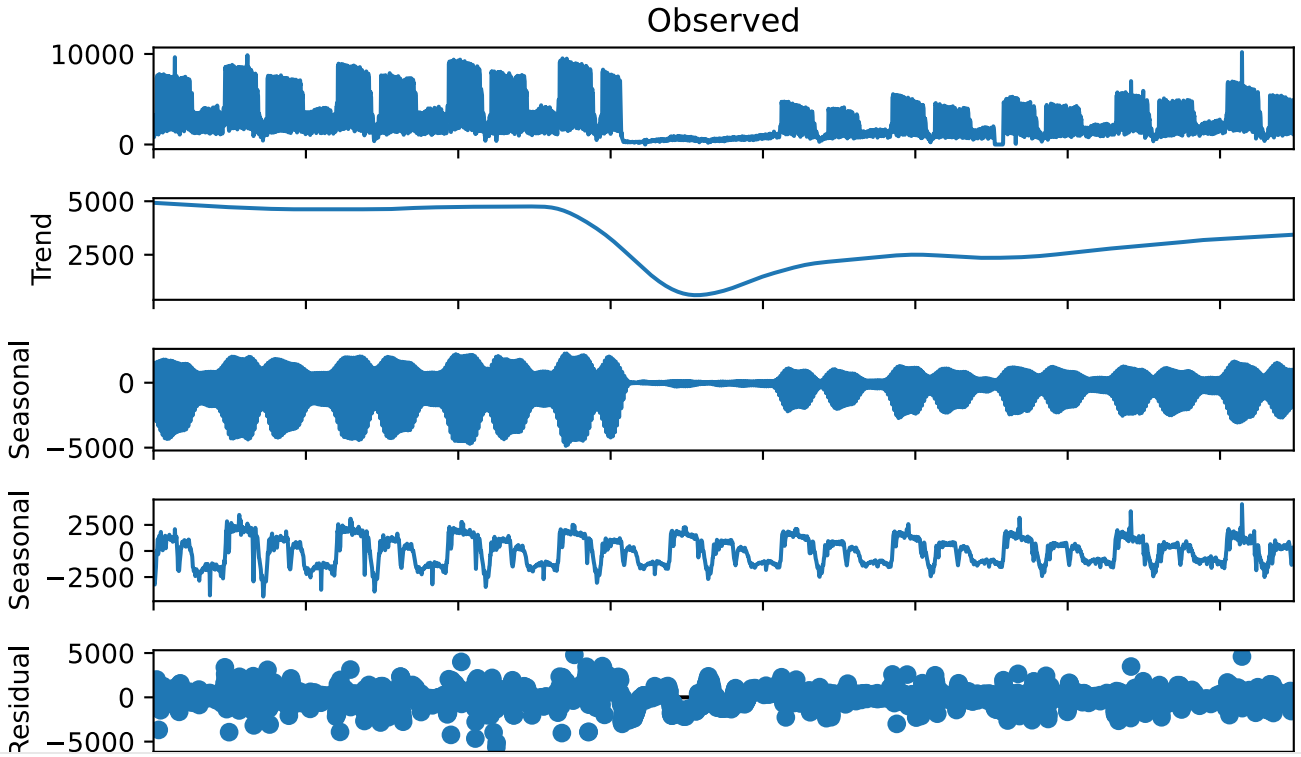
Monthly Variation(s) for UIC-Halsted Stop

```
month_order = ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
               "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
uic_month = uic.with_columns(pl.col("date").dt.strftime("%b").alias("month"))
ax = sns.boxplot(x=uic_month["month"], y=uic_month["rides"], order=month_order, color=CTA_BLUE)
ax.set_xlabel("")
ax.set_ylim(bottom=0)
ax.set_title("Ridership by Month — UIC-Halsted")
plt.tight_layout()
```



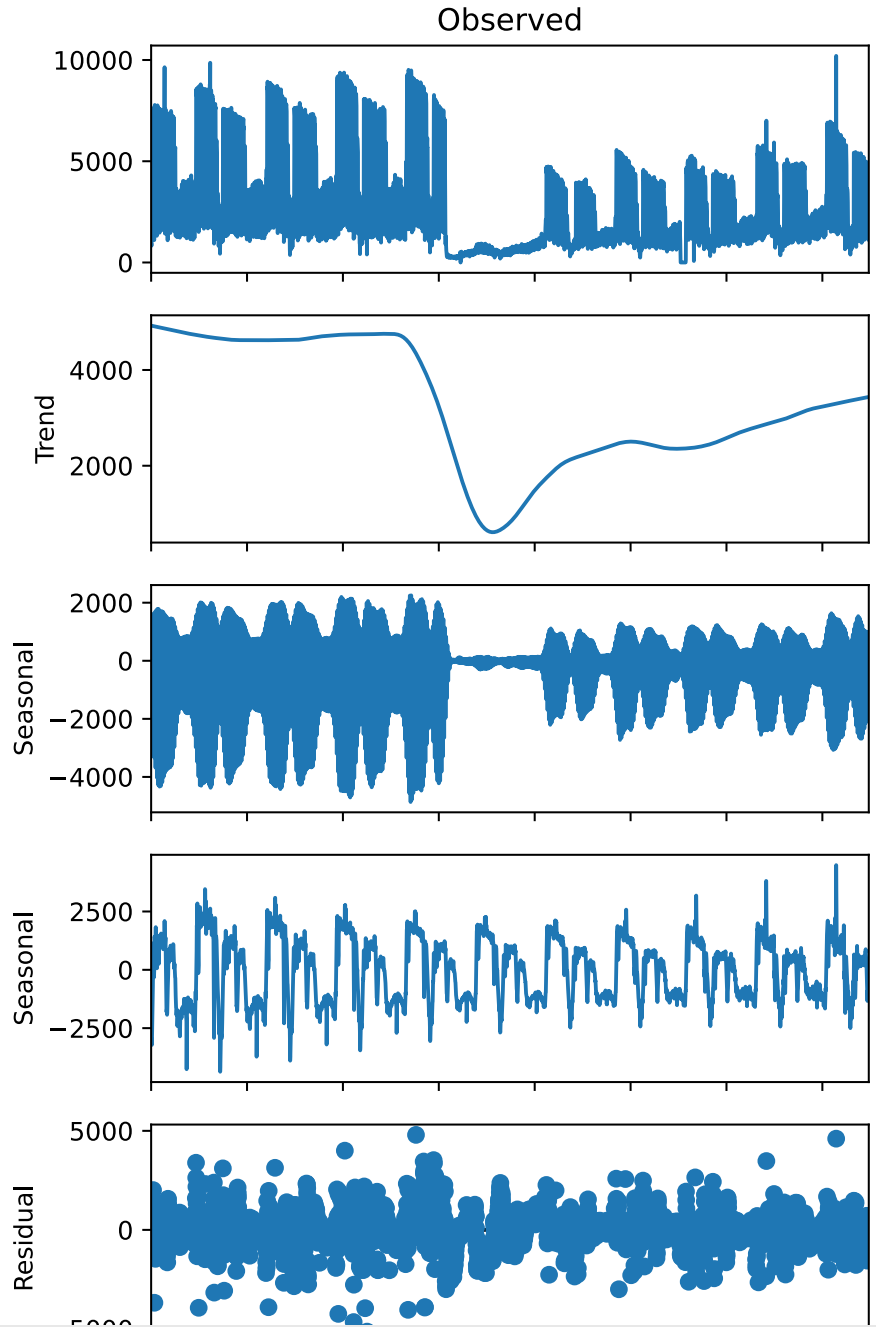
MSTL: Multiple Seasonal Decomposition

```
1 from statsmodels.tsa.seasonal import MSTL
2
3 mstl = MSTL(rides, periods=(7, 365)).fit()
4 fig = mstl.plot()
5 fig.set_size_inches(7, 4.6)
6 fig.get_axes()[-1].set_xlabel("Day # (since Jan 2016)")
7 plt.tight_layout()
```



Understanding the Decomposition

- **Trend**: steady through ~2019, collapse in spring 2020 (COVID), gradual recovery
- **Weekly seasonality**: weekday vs weekend ridership
- **Annual seasonality**: dips in January, summer, and December (academic calendar)
- **Residuals**: large spikes or dips mark unusual days — holidays,



A Different Station: Washington/Wabash

Load Washington/Wabash

```python
CTA_ORANGE = "#F9461C"
wabash = (
    cta.filter(pl.col("stationname") == "Washington/Wabash")
    .with_columns(
        pl.col("date").str.to_date("%m/%d/%Y"),
        pl.col("rides").str.replace_all(",", "").cast(pl.Int64),
    ).sort("date").filter(pl.col("date") >= pl.date(2016, 1, 1))
)
wabash.sample(5, seed=42).sort("date")
```

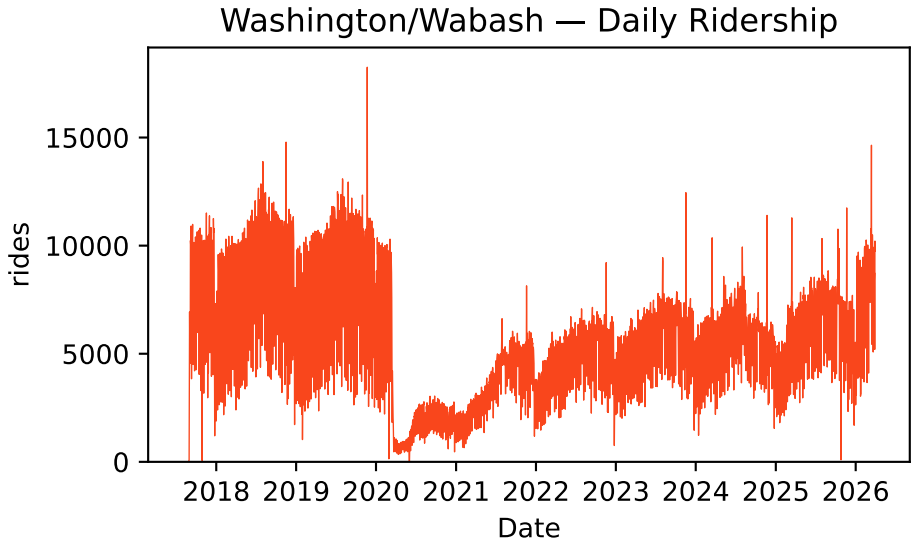
shape: (5, 5)

station_id	stationname	date	daytype	rides
i64	str	date	str	i64
41700	"Washington/Wabash"	2020-05-05	"W"	849
41700	"Washington/Wabash"	2023-08-28	"W"	6363
41700	"Washington/Wabash"	2024-06-16	"U"	4376

41700	"Washington/Wabash"	2024-08-18	"U"	4216
41700	"Washington/Wabash"	2026-02-07	"A"	5544

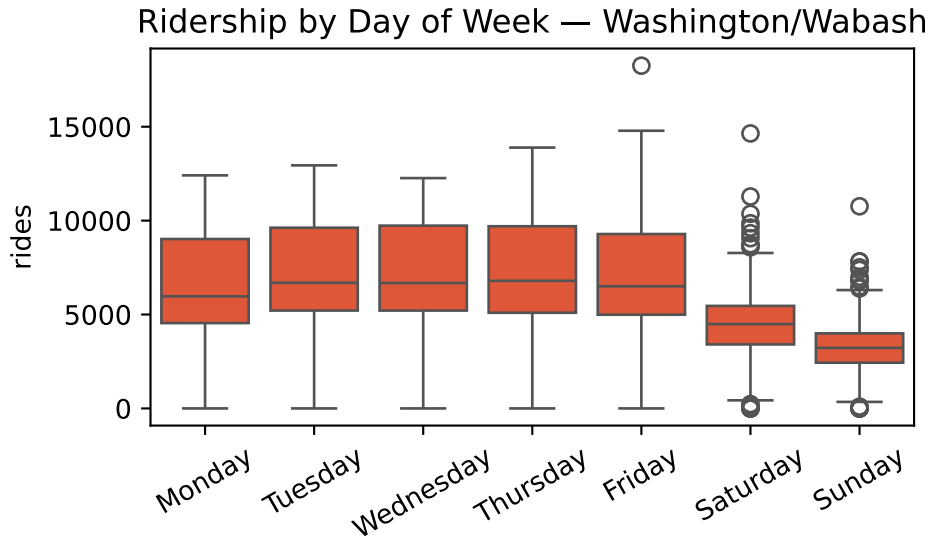
Washington/Wabash: Ridership Over Time

```
ax = sns.lineplot(x=wabash["date"], y=wabash["rides"], linewidth=0.5, color=CTA_ORANGE)
ax.set_xlabel("Date")
ax.set_ylim(bottom=0)
ax.set_title("Washington/Wabash — Daily Ridership")
plt.tight_layout()
```



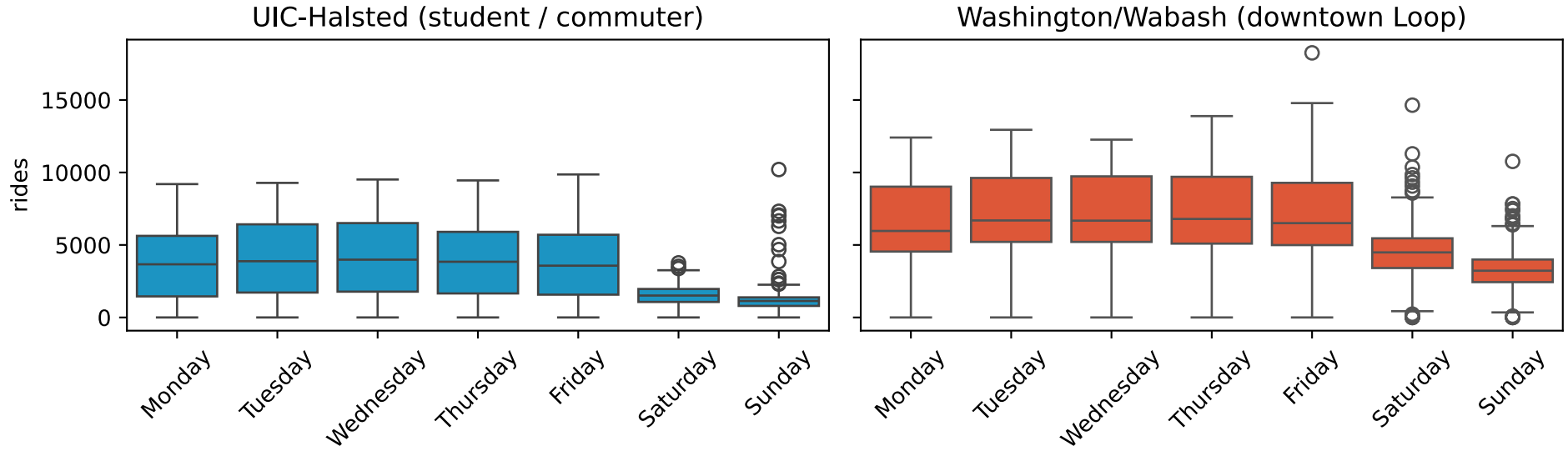
Washington/Wabash: Weekday Patterns

```
wabash_dow = wabash.with_columns(pl.col("date").dt.strftime("%A").alias("day_name"))
ax = sns.boxplot(x=wabash_dow["day_name"], y=wabash_dow["rides"], order=day_order, color=CTA_ORANGE)
ax.set_xlabel("")
ax.tick_params(axis="x", rotation=30)
ax.set_title("Ridership by Day of Week — Washington/Wabash")
plt.tight_layout()
```



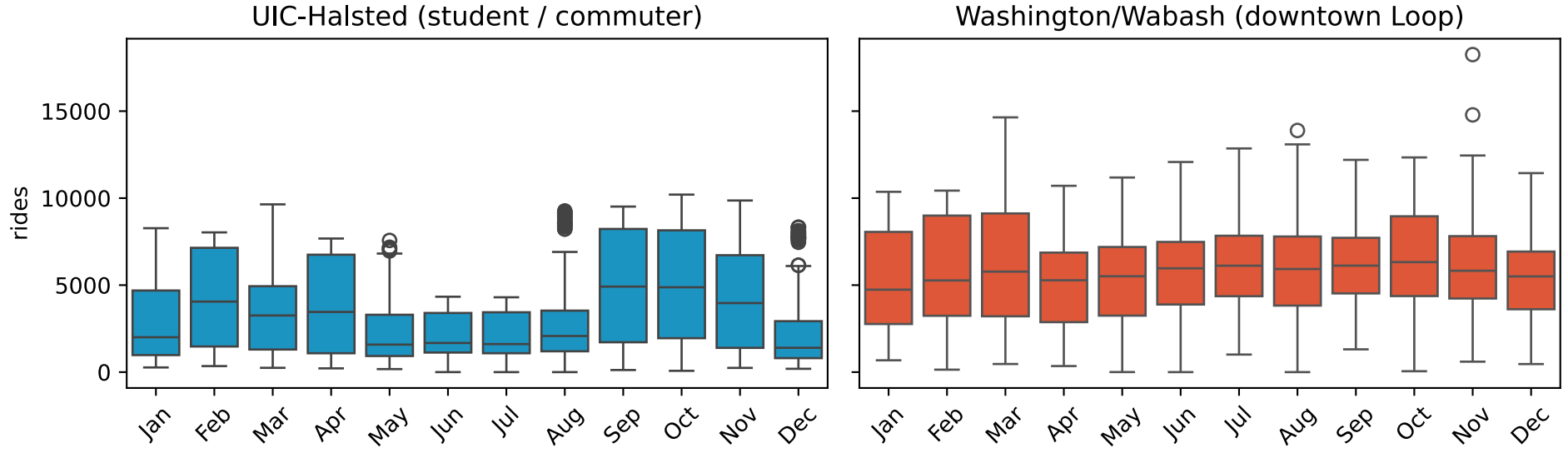
Two Stations, Two Rhythms? (Weekday)

```
fig, axes = plt.subplots(1, 2, figsize=(10, 3.09), sharey=True)
sns.boxplot(x=uic_dow["day_name"], y=uic_dow["rides"], order=day_order, color=CTA_BLUE, ax=axes[0])
sns.boxplot(x=wabash_dow["day_name"], y=wabash_dow["rides"], order=day_order, color=CTA_ORANGE, ax=axes[1])
axes[0].set_title("UIC-Halsted (student / commuter)")
axes[1].set_title("Washington/Wabash (downtown Loop)")
for ax in axes: ax.set_xlabel(""); ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
```



Two Stations, Two Rhythms? (Monthly)

```
wabash_month = wabash.with_columns(pl.col("date").dt.strftime("%b").alias("month"))
fig, axes = plt.subplots(1, 2, figsize=(10, 3.09), sharey=True)
sns.boxplot(x=uic_month["month"], y=uic_month["rides"], order=month_order, color=CTA_BLUE, ax=axes[0])
sns.boxplot(x=wabash_month["month"], y=wabash_month["rides"], order=month_order, color=CTA_ORANGE, ax=axes[1])
axes[0].set_title("UIC-Halsted (student / commuter)")
axes[1].set_title("Washington/Wabash (downtown Loop)")
for ax in axes: ax.set_xlabel(""); ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
```



Intuition

- **Weekday : weekend ratio**
  - the Loop has more weekend traffic (shopping, tourism, events, etc.)
- **Academic calendar**
  - UIC dips in January, summer, and December
  - the Loop's annual seasonal swing is weaker
- **History** — Washington/Wabash only opened in **Aug 2017**
  - (replaced Madison/Wabash and Randolph/Wabash)
  - its series is much shorter

More Forecasting Models

Forecasting Models: Overview

May be useful when working with more complex, intricate time series.

- **AR(p)** — predict from the last *p* observations
- **MA(q)** — predict from the last *q* forecast errors
- **ARMA(p, q)** — combines AR and MA; requires a stationary series
- **ARIMA(p, d, q)** — ARMA after *d* rounds of differencing; handles trend
- **SARIMA(p, d, q)(P, D, Q)m** — adds seasonal AR, differencing, MA

Bayesian Time Series

Why Bayesian?

- Classical models usually just give a **single point forecast**
    - a Bayesian model returns a **full posterior distribution** for every quantity
    - every estimate and prediction comes with a **credible interval**
- **Priors** let you encode what you already know
    - e.g. ridership can't be negative
    - weekends will be different
- Plays nicely with small data, missing days

A Bayesian Ridership Model

Let's start by modeling each day as **intercept + weekday effect + month effect + noise**, and let the data update our beliefs.

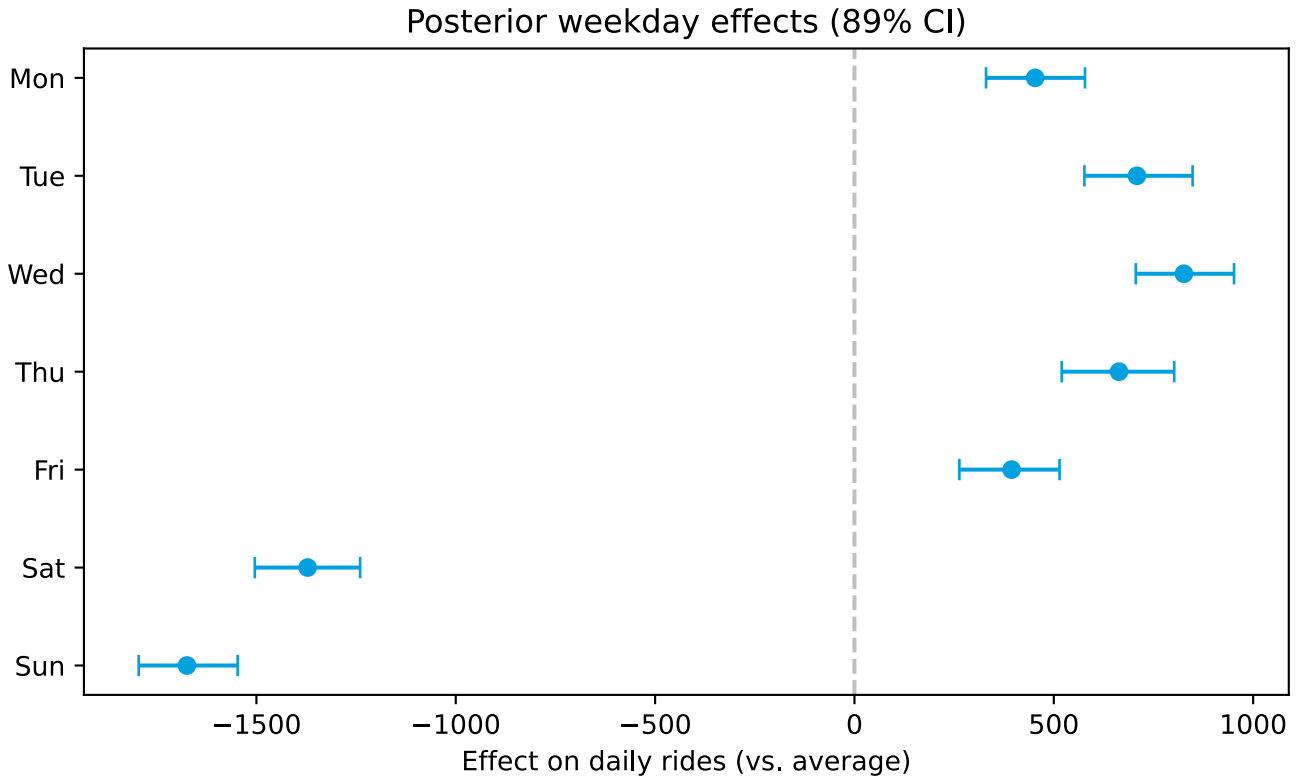
```python
import numpy as np
import pymc as pm

recent = uic.filter(pl.col("date") >= pl.date(2023, 1, 1)).with_columns(
    (pl.col("date").dt.weekday() - 1).alias("dow"),   # 0=Mon … 6=Sun
    (pl.col("date").dt.month() - 1).alias("month"),    # 0=Jan … 11=Dec
)
dow   = recent["dow"].to_numpy()
month = recent["month"].to_numpy()
y     = recent["rides"].to_numpy().astype(float)

with pm.Model() as model:
    intercept = pm.Normal("intercept", mu=y.mean(), sigma=2000)
    dow_eff   = pm.ZeroSumNormal("dow_eff", sigma=1500, shape=7)
    month_eff = pm.ZeroSumNormal("month_eff", sigma=1500, shape=12)
    sigma     = pm.HalfNormal("sigma", 2000)
    mu = intercept + dow_eff[dow] + month_eff[month]
    pm.Normal("obs", mu=mu, sigma=sigma, observed=y)
```

Posterior: Weekday Effects

```
post = idata.posterior
dow_s = post["dow_eff"].values.reshape(-1, 7)
means = dow_s.mean(0); lo, hi = np.percentile(dow_s, [5.5, 94.5], 0)
fig, ax = plt.subplots(figsize=(7, 4.3))
ax.errorbar(means, range(7), xerr=[means-lo, hi-means], fmt="o", color=CTA_BLUE, capsize=4)
ax.axvline(0, color="gray", linestyle="--", alpha=0.5)
ax.set_yticks(range(7)); ax.set_yticklabels(["Mon","Tue","Wed","Thu","Fri","Sat","Sun"]); ax.invert_yaxis()
ax.set(xlabel="Effect on daily rides (vs. average)", title="Posterior weekday effects (89% CI)")
plt.tight_layout()
```


Posterior weekday effects (89% CI)
Mon
Tue
Wed
Thu
Fri
Sat
Sun
−1500
−1000
−500
0
500
1000
Effect on daily rides (vs. average)

Using the Model to Predict

Fitting and Forecasting

Each posterior draw is one **plausible version of the world**.

- To predict a new day:
    - push **(weekday, month)** through *every* draw (every version of the world)
- That gives a **distribution** of outcomes — not just a single number!
- The **spread** provides uncertainty
    - only use **mean** if asked 🙂
- e.g. “how many rides on a *Tuesday in December*?” → let’s look!

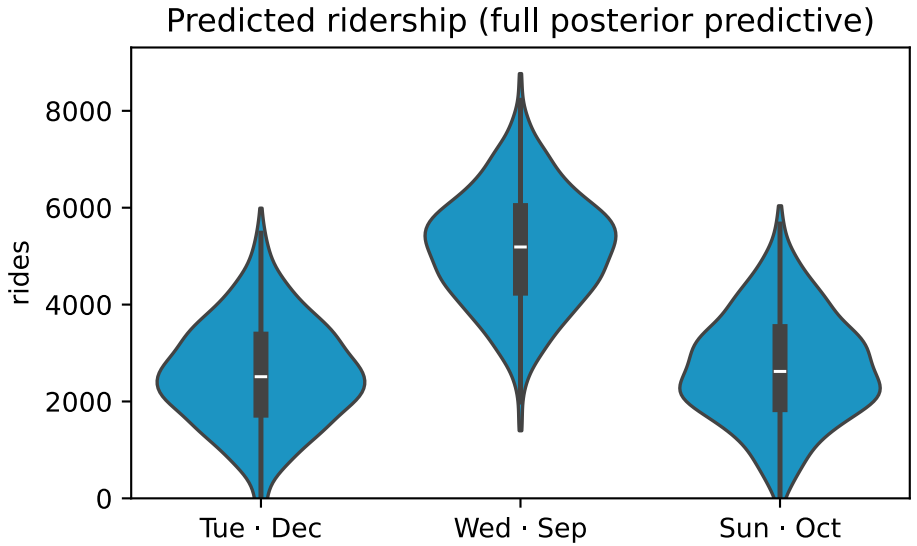
Predicting a Given Day

```
intercept_s = post["intercept"].values.flatten()
month_s     = post["month_eff"].values.reshape(-1, 12)
sigma_s     = post["sigma"].values.flatten()
rng = np.random.default_rng(0)

def predict(day, mon):
    mu = intercept_s + dow_s[:, day] + month_s[:, mon]
    return rng.normal(mu, sigma_s)
```

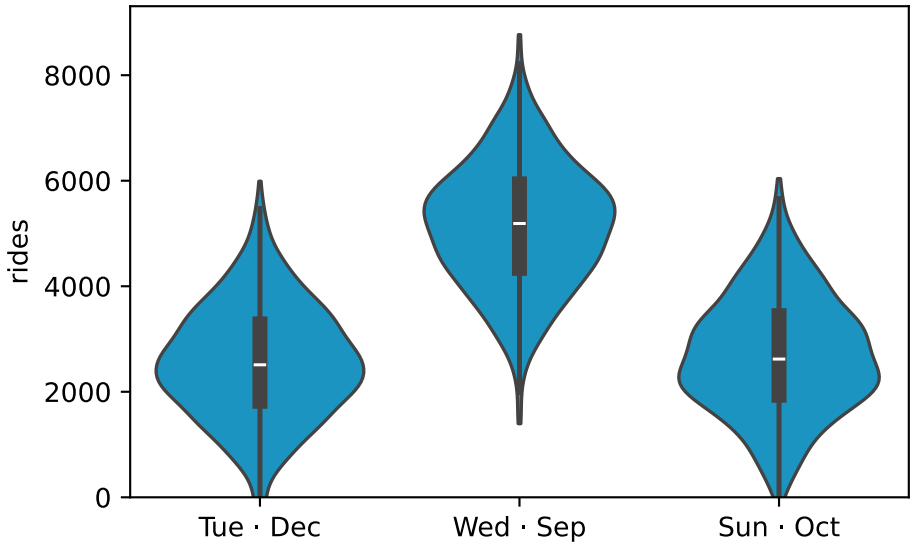
Predicting a Given Day

```python
scenarios = {"Tue · Dec": (1, 11), "Wed · Sep": (2, 8), "Sun · Oct": (6, 9)}
frames = []
for k, v in scenarios.items():
    r = predict(*v)
    frames.append(pl.DataFrame({"scenario": [k] * len(r), "rides": r}))
draws = pl.concat(frames)
ax = sns.violinplot(x=draws["scenario"], y=draws["rides"], color=CTA_BLUE, cut=0)
ax.set_ylim(bottom=0); ax.set_xlabel("")
ax.set_title("Predicted ridership (full posterior predictive)")
plt.tight_layout()
```



Reading the Predictions

- **Tue in December**: ≈ 2,530 rides, 89% CI: 480 – 4,620
- **Wed in September**: ≈ 5,150 rides, 89% CI: 2,950 – 7,260
- **Sun in October**: ≈ 2,680 rides, 89% CI: 610 – 4,860
- Intervals are wide (lots of **day-to-day noise** remains after weekday, month)



Visualizing Intervals

Ways to Visualize Intervals

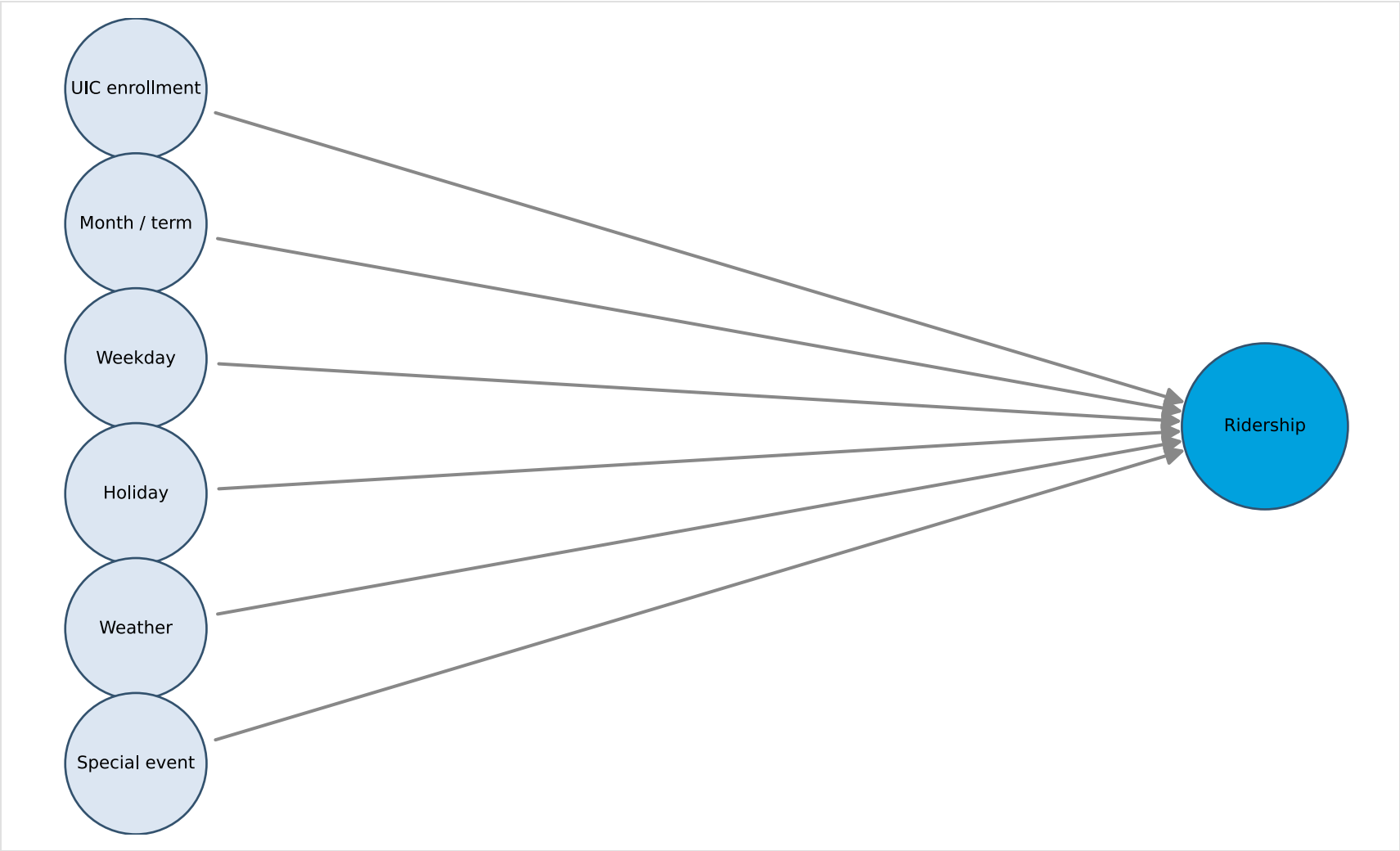
- TK
- TK1
- TK2

Another Approach: Ignore Time

So far, the date was essentially the model. But ridership responds to factors (dare we say **causes??**), which can be identified and measured.

- Treat each day as an independent **observation** (a “unit”), not a step in a sequence.
- Predict from **covariates**: enrollment, weekday, holiday, special event, weather, etc…
    - What else comes to mind?
- This is a **causal / regression** framing
    - a preview of how we’ll model

A Simple Causal DAG for Ridership

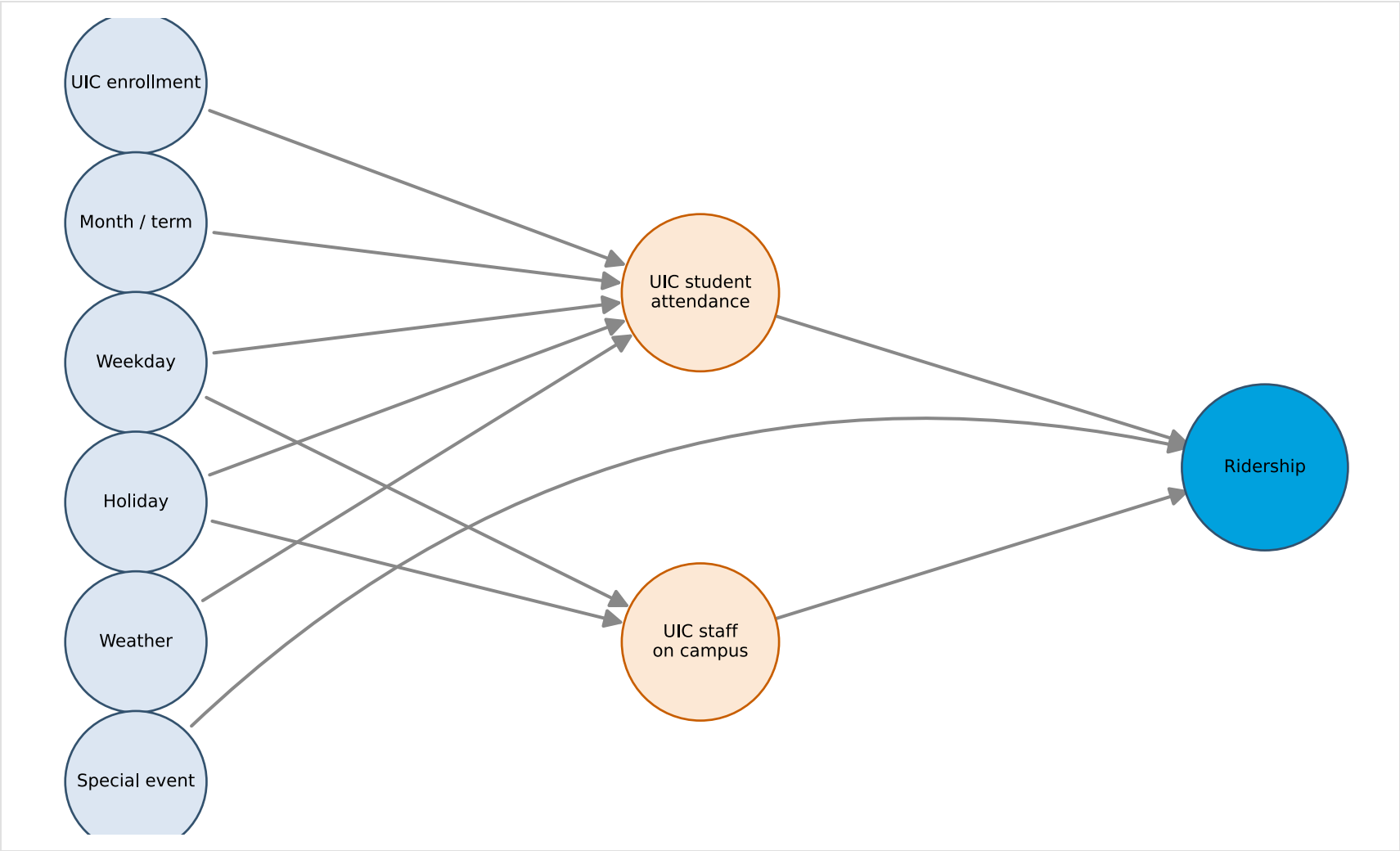


A Causal DAG for Ridership

What drives daily ridership at a station? Not always directly — some causes work through **intermediate factors**.

- **UIC enrollment** + **month/term** + **weather** → **UIC student attendance**
- **Weekday** + **holiday** → student attendance, **UIC staff on campus**
    - note the *academic calendar* affects students but **not** staff
- **Special event** like concerts, games, speakers → directly boosts ridership (could also do this as a hierarchy)
- **Ridership** ← student attendance + staff on campus + special events + etc.

A Hierarchical Causal DAG for Ridership



Sources

1. GitHub source: https://github.com/jackbandy/data-science-fun/blob/main/docs/slides/week6.qmd.
2. Last modified and compiled June 23, 22:13h Central (Chicago) time.
3. Time series decomposition: Jason Brownlee, *How to Decompose Time Series Data into Trend and Seasonality*, Machine Learning Mastery.
4. Airline passengers dataset:
    - Box, G.E.P., Jenkins, G.M. (1976). *Time Series Analysis: Forecasting and Control.* Holden-Day. ISBN 978-0-8162-1104-3. (Internet Archive scan) (5th edition via Wiley)
    - Dataset in R's base `datasets` package: `AirPassengers` — official R documentation with column definitions and citation
    - Rdatasets mirror (CSV download): https://vincentarelbundock.github.io/Rdatasets/doc/datasets/AirPassengers.html
5. CTA ridership data: Chicago Data Portal — CTA Ridership: Station Entries — Daily Totals.
6. Slides developed using materials from Elena Zheleva and Gonzalo Bello Lander, the Berkeley DS 100 team, Marine Carpuat, and Brian Ziebart.
7. Slide deck built with Quarto revealjs.
8. Title font is Big Shoulders; Body font is Libre Franklin.

Appendix

ACF and PACF

Use ACF and PACF plots to choose p and q from data. Two tools for choosing model order from data:

- **ACF (Autocorrelation Function)** — correlation between y_t and y_{t-k} for each lag k. Use to choose MA order q: ACF "cuts off" after lag q.
- **PACF (Partial Autocorrelation Function)** — correlation at lag k after removing the effects explained by shorter lags. Use to choose AR order p: PACF "cuts off" after lag p.

Reading ACF and PACF Plots

Each bar = correlation at that lag; shaded band = 95% confidence interval.

- Bars **within** the band → not significant (treat as zero)
- **ACF decays slowly** → series may not be stationary — try differencing first
- **ACF cuts off** sharply after lag q → suggests **MA(q)**
- **PACF cuts off** sharply after lag p → suggests **AR(p)**
- **Both decay gradually** → suggests **ARMA(p, q)** — compare with AIC / BIC

ACF and PACF: Airline Passengers

```python
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 4.3))
plot_acf(values, lags=30, ax=ax1, title="ACF — Airline Passengers")
plot_pacf(values, lags=30, ax=ax2, title="PACF — Airline Passengers")
plt.tight_layout()
```